

Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem

Marco Dorigo, *Senior Member, IEEE*, and Luca Maria Gambardella, *Member, IEEE*

Abstract—This paper introduces the ant colony system (ACS), a distributed algorithm that is applied to the traveling salesman problem (TSP). In the ACS, a set of cooperating agents called ants cooperate to find good solutions to TSP's. Ants cooperate using an indirect form of communication mediated by a pheromone they deposit on the edges of the TSP graph while building solutions. We study the ACS by running experiments to understand its operation. The results show that the ACS outperforms other nature-inspired algorithms such as simulated annealing and evolutionary computation, and we conclude comparing ACS-3-opt, a version of the ACS augmented with a local search procedure, to some of the best performing algorithms for symmetric and asymmetric TSP's.

Index Terms—Adaptive behavior, ant colony, emergent behavior, traveling salesman problem.

I. INTRODUCTION

THE natural metaphor on which ant algorithms are based is that of ant colonies. Real ants are capable of finding the shortest path from a food source to their nest [3], [22] without using visual cues [24] by exploiting pheromone information. While walking, ants deposit pheromone on the ground and follow, in probability, pheromone previously deposited by other ants. In Fig. 1, we show a way ants exploit pheromone to find a shortest path between two points.

Consider Fig. 1(a): ants arrive at a decision point in which they have to decide whether to turn left or right. Since they have no clue about which is the best choice, they choose randomly. It can be expected that, on average, half of the ants decide to turn left and the other half to turn right. This happens both to ants moving from left to right (those whose name begins with an L) and to those moving from right to left (name begins with an R). Fig. 1(b) and (c) shows what happens in the immediately following instants, supposing that all ants walk at approximately the same speed. The number of dashed lines is roughly proportional to the amount of pheromone that the ants have deposited on the ground. Since the lower path is shorter than the upper one, more ants will visit it on average, and therefore pheromone accumulates faster. After a short transitory period the difference in the amount of pheromone on the two paths is sufficiently large so as to influence the decision of new ants coming into the system [this is shown by

Fig. 1(d)]. From now on, new ants will prefer in probability to choose the lower path, since at the decision point they perceive a greater amount of pheromone on the lower path. This in turn increases, with a positive feedback effect, the number of ants choosing the lower, and shorter, path. Very soon all ants will be using the shorter path.

The above behavior of real ants has inspired *ant system*, an algorithm in which a set of artificial ants cooperate to the solution of a problem by exchanging information via pheromone deposited on graph edges. The ant system has been applied to combinatorial optimization problems such as the traveling salesman problem (TSP) [7], [8], [10], [12] and the quadratic assignment problem [32], [42].

The ant colony system (ACS), the algorithm presented in this article, builds on the previous ant system in the direction of improving efficiency when applied to symmetric and asymmetric TSP's. The main idea is that of having a set of agents, called *ants*, search in parallel for good solutions to the TSP and cooperate through pheromone-mediated indirect and global communication. Informally, each ant constructs a TSP solution in an iterative way: it adds new cities to a partial solution by exploiting both information gained from past experience and a greedy heuristic. Memory takes the form of pheromone deposited by ants on TSP edges, while heuristic information is simply given by the edge's length.

The main novel idea introduced by ant algorithms, which will be discussed in the remainder of the paper, is the synergistic use of cooperation among many relatively simple agents which communicate by distributed memory implemented as pheromone deposited on edges of a graph.

This paper is organized as follows. Section II puts the ACS in context by describing ant system, the progenitor of the ACS. Section III introduces the ACS. Section IV is dedicated to the study of some characteristics of the ACS: We study how pheromone changes at run time, estimate the optimal number of ants to be used, observe the effects of pheromone-mediated cooperation, and evaluate the role that pheromone and the greedy heuristic have in ACS performance. Section V provides an overview of results on a set of standard test problems and comparisons of the ACS with well-known general purpose algorithms like evolutionary computation and simulated annealing. In Section VI we add local optimization to the ACS, obtaining a new algorithm called ACS-3-opt. This algorithm is compared favorably with the winner of the First International Contest on Evolutionary Optimization [5] on asymmetric TSP (ATSP) problems (see Fig. 2), while it yields a slightly worse performance on TSP problems. Section VII is

Manuscript received October 7, 1996; revised January 18, 1997 and February 3, 1997. This work was supported by the Swiss National Science Fund Contract 21-45 653.95.

M. Dorigo is with IRIDIA, Université Libre de Bruxelles, 1050 Bruxelles, Belgium (e-mail: mdorigo@ulb.ac.be).

L. M. Gambardella is with IDSIA, 6900 Lugano, Switzerland.

Publisher Item Identifier S 1089-778X(97)03303-1.

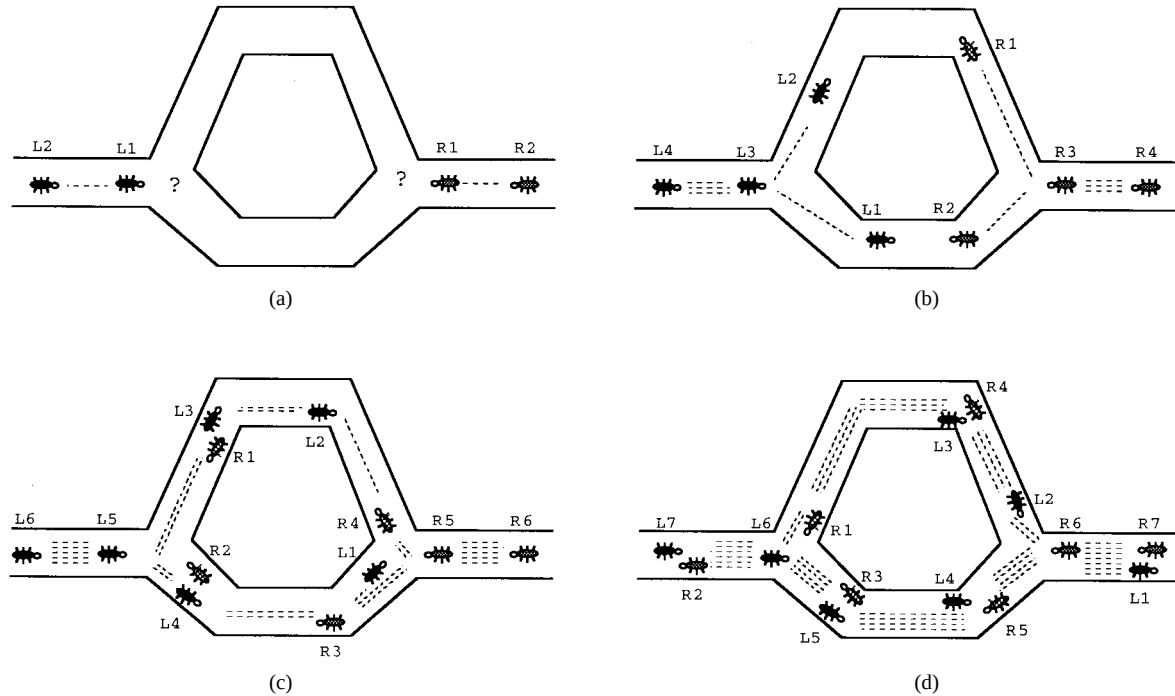


Fig. 1. How real ants find a shortest path. (a) Ants arrive at a decision point. (b) Some ants choose the upper path and some the lower path. The choice is random. (c) Since ants move at approximately a constant speed, the ants which choose the lower, shorter, path reach the opposite decision point faster than those which choose the upper, longer, path. (d) Pheromone accumulates at a higher rate on the shorter path. The number of dashed lines is approximately proportional to the amount of pheromone deposited by ants.

TSP

Let $V = \{a, \dots, z\}$ be a set of cities, $A = \{(r,s): r,s \in V\}$ be the edge set, and $\delta(r,s) = \delta(s,r)$ be a cost measure associated with edge $(r,s) \in A$.

The TSP is the problem of finding a minimal cost closed tour that visits each city once.

In the case cities $r \in V$ are given by their coordinates (x_r, y_r) and $\delta(r,s)$ is the Euclidean distance between r and s , then we have an Euclidean TSP.

ATSP

If $\delta(r,s) \neq \delta(s,r)$ for at least some (r,s) then the TSP becomes an asymmetric TSP (ATSP).

Fig. 2. The traveling salesman problem.

dedicated to the discussion of the main characteristics of the ACS and indicates directions for further research.

II. BACKGROUND

Ant system [10] is the progenitor of all our research efforts with ant algorithms and was first applied to the TSP, which is defined in Fig. 2.

Ant system utilizes a graph representation which is the same as that defined in Fig. 2, augmented as follows: in addition to the cost measure $\delta(r,s)$, each edge (r,s) has also a desirability measure $\tau(r,s)$, called *pheromone*, which is updated at run time by artificial ants (*ants* for short). When ant system is applied to symmetric instances of the TSP, $\tau(r,s) = \tau(s,r)$, but when it is applied to asymmetric instances it is possible that $\tau(r,s) \neq \tau(s,r)$.

Informally, ant system works as follows. Each ant generates a complete tour by choosing the cities according to a probabilistic *state transition rule*; ants prefer to move to cities which are connected by short edges with a high amount of pheromone. Once all ants have completed their tours a *global pheromone updating rule* (global updating rule, for short) is applied; a fraction of the pheromone evaporates on all edges (edges that are not refreshed become less desirable), and then each ant deposits an amount of pheromone on edges which belong to its tour in proportion to how short its tour was (in other words, edges which belong to many short tours are the edges which receive the greater amount of pheromone). The process is then iterated.

The state transition rule used by ant system, called a *random-proportional rule*, is given by (1), which gives the

```

Initialize
Loop /* at this level each loop is called an iteration */
    Each ant is positioned on a starting node
    Loop /* at this level each loop is called a step */
        Each ant applies a state transition rule to incrementally build a solution
        and a local pheromone updating rule
    Until all ants have built a complete solution
    A global pheromone updating rule is applied
Until End_condition

```

Fig. 3. The ACS algorithm.

probability with which ant k in city r chooses to move to the city s

$$p_k(r, s) = \begin{cases} \frac{[\tau(r, s)] \cdot [\eta(r, s)]^\beta}{\sum_{u \in J_k(r)} [\tau(r, u)] \cdot [\eta(r, u)]^\beta}, & \text{if } s \in J_k(r) \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

where τ is the pheromone, $\eta = 1/\delta$ is the inverse of the distance $\delta(r, s)$, $J_k(r)$ is the set of cities that remain to be visited by ant k positioned on city r (to make the solution feasible), and β is a parameter which determines the relative importance of pheromone versus distance ($\beta > 0$).

In (1) we multiply the pheromone on edge (r, s) by the corresponding heuristic value $\eta(r, s)$. In this way we favor the choice of edges which are shorter and which have a greater amount of pheromone.

In ant system, the global updating rule is implemented as follows. Once all ants have built their tours, pheromone is updated on all edges according to

$$\tau(r, s) \leftarrow (1 - \alpha) \cdot \tau(r, s) + \sum_{k=1}^m \Delta\tau_k(r, s) \quad (2)$$

where

$$\Delta\tau_k(r, s) = \begin{cases} \frac{1}{L_k}, & \text{if } (r, s) \in \text{tour done by ant } k \\ 0, & \text{otherwise} \end{cases}$$

$0 < \alpha < 1$ is a pheromone decay parameter, L_k is the length of the tour performed by ant k , and m is the number of ants.

Pheromone updating is intended to allocate a greater amount of pheromone to shorter tours. In a sense, this is similar to a reinforcement learning scheme [2], [26] in which better solutions get a higher reinforcement (as happens, for example, in genetic algorithms under proportional selection). The pheromone updating formula was meant to simulate the change in the amount of pheromone due to both the addition of new pheromone deposited by ants on the visited edges and to pheromone evaporation.

Pheromone placed on the edges plays the role of a distributed long-term memory: this memory is not stored locally within the individual ants, but is distributed on the edges of the graph. This allows an indirect form of communication called *stigmergy* [9], [23]. The interested reader will find a full description of ant system, of its biological motivations, and computational results in [12].

Although ant system was useful for discovering good or optimal solutions for small TSP's (up to 30 cities), the time required to find such results made it infeasible for larger problems. We devised three main changes to improve its performance which led to the definition of the ACS, presented in the next section.

III. ACS

The ACS differs from the previous ant system because of three main aspects: i) the state transition rule provides a direct way to balance between exploration of new edges and exploitation of *a priori* and accumulated knowledge about the problem, ii) the global updating rule is applied only to edges which belong to the best ant tour, and iii) while ants construct a solution a *local pheromone updating rule* (local updating rule, for short) is applied.

Informally, the ACS works as follows: m ants are initially positioned on n cities chosen according to some initialization rule (e.g., randomly). Each ant builds a tour (i.e., a feasible solution to the TSP) by repeatedly applying a stochastic greedy rule (the state transition rule). While constructing its tour, an ant also modifies the amount of pheromone on the visited edges by applying the local updating rule. Once all ants have terminated their tour, the amount of pheromone on edges is modified again (by applying the global updating rule). As was the case in ant system, ants are guided, in building their tours, by both heuristic information (they prefer to choose short edges) and by pheromone information. An edge with a high amount of pheromone is a very desirable choice. The pheromone updating rules are designed so that they tend to give more pheromone to edges which should be visited by ants. The ACS algorithm is reported in Fig. 3. In the following we discuss the state transition rule, the global updating rule, and the local updating rule.

A. ACS State Transition Rule

In the ACS the state transition rule is as follows: an ant positioned on node r chooses the city s to move to by applying the rule given by (3)

$$s = \begin{cases} \arg \max_{u \in J_k(r)} \{[\tau(r, u)] \cdot [\eta(r, u)]^\beta\}, & \text{if } q \leq q_0 \quad (\text{exploitation}) \\ S, & \text{otherwise} \quad (\text{biased exploration}) \end{cases} \quad (3)$$

where q is a random number uniformly distributed in $[0 \dots 1]$, q_0 is a parameter ($0 \leq q_0 \leq 1$), and S is a random variable selected according to the probability distribution given in (1).

The state transition rule resulting from (3) and (1) is called *pseudo-random-proportional rule*. This state transition rule, as with the previous random-proportional rule, favors transitions toward nodes connected by short edges and with a large amount of pheromone. The parameter q_0 determines the relative importance of exploitation versus exploration: every time an ant in city r has to choose a city s to move to, it samples a random number $0 \leq q \leq 1$. If $q \leq q_0$ then the best edge, according to (3), is chosen (exploitation), otherwise an edge is chosen according to (1) (biased exploration).

B. ACS Global Updating Rule

In ACS only the globally best ant (i.e., the ant which constructed the shortest tour from the beginning of the trial) is allowed to deposit pheromone. This choice, together with the use of the pseudo-random-proportional rule, is intended to make the search more directed: ants search in a neighborhood of the best tour found up to the current iteration of the algorithm. Global updating is performed after all ants have completed their tours. The pheromone level is updated by applying the global updating rule of (4)

$$\tau(r, s) \leftarrow (1 - \alpha) \cdot \tau(r, s) + \alpha \cdot \Delta\tau(r, s) \quad (4)$$

where

$$\Delta\tau(r, s) = \begin{cases} (L_{gb})^{-1}, & \text{if } (r, s) \in \text{global-best-tour} \\ 0, & \text{otherwise} \end{cases}$$

$0 < \alpha < 1$ is the pheromone decay parameter, and L_{gb} is the length of the globally best tour from the beginning of the trial. As was the case in ant system, global updating is intended to provide a greater amount of pheromone to shorter tours. Equation (4) dictates that only those edges belonging to the globally best tour will receive reinforcement. We also tested another type of global updating rule, called *iteration-best*, as opposed to the above called *global-best*, which instead used L_{ib} (the length of the best tour in the current iteration of the trial), in (4). Also, with iteration-best the edges which receive reinforcement are those belonging to the best tour of the current iteration. Experiments have shown that the difference between the two schemes is minimal, with a slight preference for global-best, which is therefore used in the following experiments.

C. ACS Local Updating Rule

While building a solution (i.e., a tour) of the TSP, ants visit edges and change their pheromone level by applying the local updating rule of (5)

$$\tau(r, s) \leftarrow (1 - \rho) \cdot \tau(r, s) + \rho \cdot \Delta\tau(r, s) \quad (5)$$

where $0 < \rho < 1$ is a parameter.

We have experimented with three values for the term $\Delta\tau(r, s)$. The first choice was loosely inspired by Q-learning [40], an algorithm developed to solve reinforcement learning problems [26]. Such problems are faced by an agent that must

learn the best action to perform in each possible state in which it finds itself, using as the sole learning information a scalar number which represents an evaluation of the state entered after it has performed the chosen action. Q-learning is an algorithm which allows an agent to learn such an optimal policy by the recursive application of a rule similar to that in (5), in which the term $\Delta\tau(r, s)$ is set to the discounted evaluation of the next state value. Since the problem our ants have to solve is similar to a reinforcement learning problem (ants have to learn which city to move to as a function of their current location), we set [19] $\Delta\tau(r, s) = \gamma \cdot \max_{z \in J_k(s)} \tau(s, z)$, which is exactly the same formula used in Q-learning ($0 \leq \gamma < 1$ is a parameter). The other two choices were: i) we set $\Delta\tau(r, s) = \tau_0$, where τ_0 is the initial pheromone level, and ii) we set $\Delta\tau(r, s) = 0$. Finally, we also ran experiments in which local updating was not applied (i.e., the local updating rule is not used, as was the case in ant system).

Results obtained running experiments (see Table I) on a set of five randomly generated 50-city TSP's [13], on the Oliver30 symmetric TSP [41] and the ry48p asymmetric TSP [35], essentially suggest that local updating is definitely useful and that the local updating rule with $\Delta\tau(r, s) = 0$ yields worse performance than local updating with $\Delta\tau(r, s) = \tau_0$ or with $\Delta\tau(r, s) = \gamma \cdot \max_{z \in J_k(s)} \tau(s, z)$. The ACS with $\Delta\tau(r, s) = \gamma \cdot \max_{z \in J_k(s)} \tau(s, z)$, which we have called Ant-Q in [11] and [19], and the ACS with $\Delta\tau(r, s) = \tau_0$, called simply ACS hereafter, resulted to be the two best performing algorithms, with a similar performance level. Since the ACS local updating rule requires less computation than Ant-Q, we chose to focus attention on the ACS, which will be used to run the experiments presented in the rest of this paper.

As will be discussed in Section IV-A, the role of the ACS local updating rule is to shuffle the tours, so that the early cities in one ant's tour may be explored later in other ants' tours. In other words, the effect of local updating is to make the desirability of edges change dynamically: every time an ant uses an edge this becomes slightly less desirable (since it loses some of its pheromone). In this way ants will make a better use of pheromone information: without local updating all ants would search in a narrow neighborhood of the best previous tour.

D. ACS Parameter Settings

In all experiments of the following sections the numeric parameters, except when indicated differently, are set to the following values: $\beta = 2, q_0 = 0.9, \alpha = \rho = 0.1, \tau_0 = (n \cdot L_{nn})^{-1}$, where L_{nn} is the tour length produced by the nearest neighbor heuristic¹ [36] and n is the number of cities. These values were obtained by a preliminary optimization phase, in which we found that the experimental optimal values of the parameters were largely independent of the problem, except for τ_0 for which, as we said, $\tau_0 = (n \cdot L_{nn})^{-1}$. The number of ants used is $m = 10$ (this choice is explained

¹To be true, any very rough approximation of the optimal tour length would suffice.

TABLE I
A COMPARISON OF LOCAL UPDATING RULES. FIFTY-CITY PROBLEMS AND OLIVER30 WERE STOPPED AFTER 2500 ITERATIONS, WHILE ry48p WAS HALTED AFTER 10 000 ITERATIONS. AVERAGES ARE OVER 25 TRIALS. RESULTS IN BOLD ARE THE BEST IN THE TABLE

	ACS			Ant-Q			ACS with $\Delta\tau(r,s)=0$			ACS without local-updating		
	average	std dev	best	average	std dev	best	average	std dev	best	average	std dev	best
City Set 1	5.88	0.05	5.84	5.88	0.05	5.84	5.97	0.09	5.85	5.96	0.09	5.84
City Set 2	6.05	0.03	5.99	6.07	0.07	5.99	6.13	0.08	6.05	6.15	0.09	6.05
City Set 3	5.58	0.01	5.57	5.59	0.05	5.57	5.72	0.12	5.57	5.68	0.14	5.57
City Set 4	5.74	0.03	5.70	5.75	0.04	5.70	5.83	0.12	5.70	5.79	0.05	5.71
City Set 5	6.18	0.01	6.17	6.18	0.01	6.17	6.29	0.11	6.17	6.27	0.09	6.17
Oliver30	424.74	2.83	423.74	424.70	2.00	423.74	427.52	5.21	423.74	427.31	3.63	423.91
ry48p	14,625	142	14,422	14,766	240	14,422	15,196	233	14,734	15,308	241	14,796

in Section IV-B). Regarding their initial positioning, ants are placed randomly, with at most one ant in each city.

IV. A STUDY OF SOME CHARACTERISTICS OF THE ACS

A. Pheromone Behavior and Its Relation to Performance

To try to understand which mechanism the ACS uses to direct the search we study how the pheromone-closeness product $[\tau(r,s)] \cdot [\eta(r,s)]^\beta$ changes at run time. Fig. 4 shows how the pheromone-closeness product changes with the number of steps while ants are building a solution² (steps refer to the inner loop in Fig. 3: the abscissa goes therefore from 1 to n , where n is the number of cities).

Let us consider three families of edges (see Fig. 4): i) those belonging to the last best tour (BE, best edges), ii) those which do not belong to the last best tour, but which did in one of the two preceding iterations (TE, testable edges), and iii) the remaining edges, that is, those that have never belonged to a best tour or have not in the last two iterations (UE, uninteresting edges). The average pheromone-closeness product is then computed as the average of pheromone-closeness values of all the edges within a family. The graph clearly shows that the ACS favors exploitation of edges in BE (BE edges are chosen with probability $q_0 = 0.9$) and exploration of edges in TE (recall that, since (3) and (1), edges with higher pheromone-closeness product have a higher probability of being explored).

An interesting aspect is that while edges are visited by ants, the application of the local updating rule (5) makes their pheromone diminish, making them less and less attractive, and therefore favoring the exploration of edges not yet visited. Local updating has the effect of lowering the pheromone on visited edges so that these become less desirable and therefore will be chosen with a lower probability by the other ants in the remaining steps of an iteration of the algorithm. As a consequence, ants never converge to a common path. This fact, which was observed experimentally, is a desirable property given that if ants explore different paths then there is a higher probability that one of them will find an improving solution

²The graph in Fig. 4 is an abstraction of graphs obtained experimentally. Examples of these are given in Fig. 5.

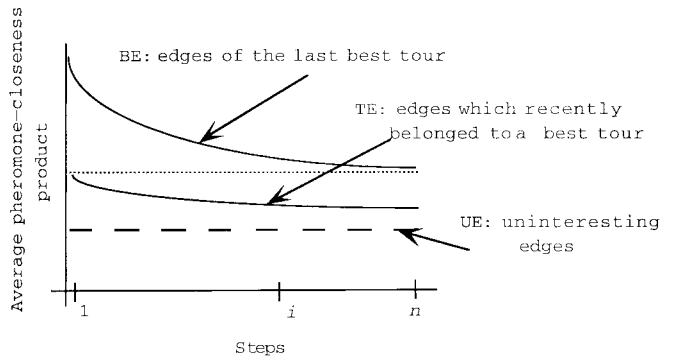


Fig. 4. Families of edges classified according to different behavior with respect to the pheromone-closeness product. The average level of the pheromone-closeness product changes in each family during one iteration of the algorithm (i.e., during n steps).

than there is in the case that they all converge to the same tour (which would make the use of m ants pointless).

Experimental observation has shown that edges in BE, when ACS achieves a good performance, will be approximately downgraded to TE after an iteration of the algorithm (i.e., one external loop in Fig. 3; see also Fig. 4) and that edges in TE will soon be downgraded to UE, unless they happen to belong to a new shortest tour.

In Fig. 5(a) and (b), we report two typical behaviors of pheromone level when the system has a good or a bad performance respectively.

B. The Optimal Number of Ants

Consider Fig. 6.³ Let $\varphi_2\tau_0$ be the average pheromone level on edges in BE just after they are updated by the global updating rule and $\varphi_1\tau_0$ the average pheromone level on edges in BE just before they are updated by the global updating rule ($\varphi_1\tau_0$ is also approximately the average pheromone level on edges in TE at the beginning of the inner loop of the algorithm). Under the hypothesis that the optimal values of φ_1 and φ_2 are known, an estimate of the optimal number of ants can be computed as follows. The local updating

³Note that this figure shows the average pheromone level, while Fig. 4 showed the average pheromone-closeness product.

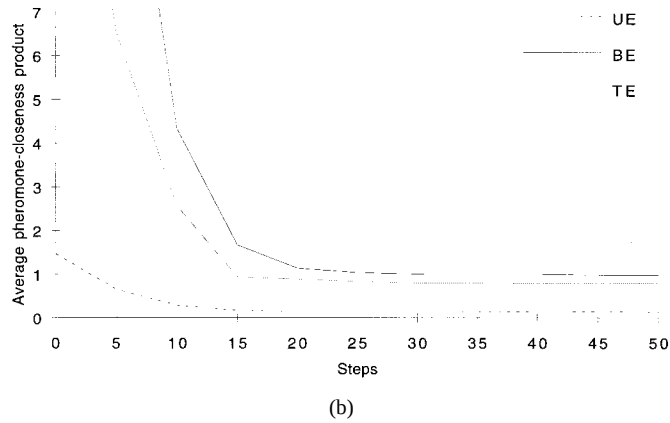
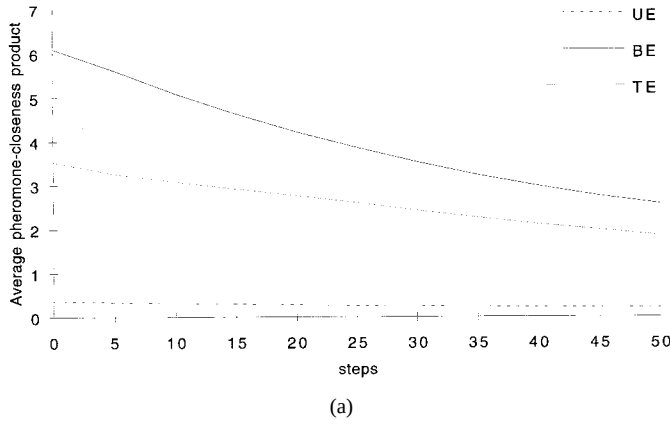


Fig. 5. Families of edges classified according to different behavior with respect to the level of the pheromone-closeness product. Problem: Eil51 [14]. (a) Pheromone-closeness behavior when the system performance is good. Best solution found after 1000 iterations: 426, $\alpha = \rho = 0.1$. (b) Pheromone-closeness behavior when the system performance is bad. Best solution found after 1000 iterations: 465, $\alpha = \rho = 0.9$.

rule is a first-order linear recurrence relation of the form $T_z = T_{z-1}(1 - \rho) + \tau_0\rho$, which has closed form given by $T_z = T_0(1 - \rho)^z - \tau_0(1 - \rho)^z + \tau_0$. Knowing that just before global updating $T_0 = \varphi_2\tau_0$ (this corresponds to the start point of the BE curve in Fig. 6) and that after all ants have built their tour and just before global updating, $T_z = \varphi_1\tau_0$ (this corresponds to the end point of the BE curve in Fig. 6), we obtain $\varphi_1 = \varphi_2(1 - \rho)^z - (1 - \rho)^z + 1$. Considering the fact that edges in BE are chosen by each ant with a probability $> q_0$, then a good approximation to the number z of ants that locally update edges in BE is given by $z = m \cdot q_0$. Substituting in the above formula we obtain the following estimate of the optimal number of ants:

$$m = \frac{\log(\varphi_1 - 1) - \log(\varphi_2 - 1)}{q_0 \cdot \log(1 - \rho)}.$$

This formula essentially shows that the optimal number of ants is a function of φ_1 and φ_2 . Unfortunately, until now, we have not been able to identify the form of the functions $\varphi_1(n)$ and $\varphi_2(n)$, which would tell how φ_1 and φ_2 change as a function of the problem dimension. Still, experimental observation shows that the ACS works well when the ratio $(\varphi_1 - 1)/(\varphi_2 - 1) \approx 0.4$, which gives $m = 10$.

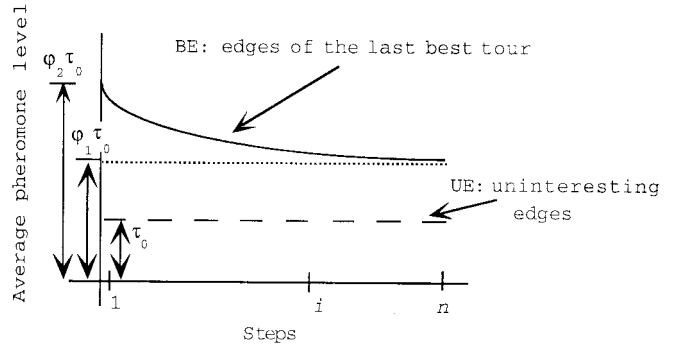


Fig. 6. Change in average pheromone level during an algorithm iteration for edges in the BE family. The average pheromone level on edges in BE starts at $\varphi_2\tau_0$ and decreases each time an ant visits an edge in BE. After one algorithm iteration, each edge in BE has been visited on average $m \cdot q_0$ times, and the final value of the pheromone level is $\varphi_1\tau_0$.

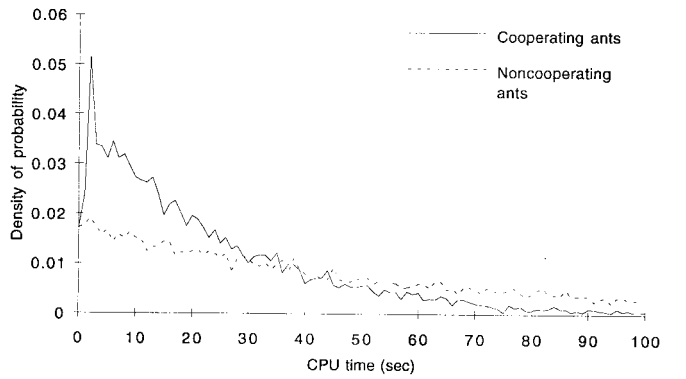


Fig. 7. Cooperation changes the probability distribution of first finishing times: cooperating ants have a higher probability to find quickly an optimal solution. Test problem: CCAO [21]. The number of ants was set to $m = 4$.

C. Cooperation Among Ants

This section presents the results of two simple experiments which show that the ACS effectively exploits pheromone-mediated cooperation. Since artificial ants cooperate by exchanging information via pheromone, to have noncooperating ants it is enough to make ants blind to pheromone. In practice this is obtained by deactivating (4) and (5) and setting the initial level of pheromone to $\tau_0 = 1$ on all edges. When comparing a colony of cooperating ants with a colony of noncooperating ants, to make the comparison fair, we use CPU time to compute performance indexes so as to discount for the higher complexity, due to pheromone updating, of the cooperative approach.

In the first experiment, the distribution of *first finishing times*, defined as the time elapsed until the first optimal solution is found, is used to compare the cooperative and the noncooperative approaches. The algorithm is run 10 000 times, and then we report on a graph the probability distribution (density of probability) of the CPU time needed to find the optimal value (e.g., if in 100 trials the optimum is found after exactly 220 iterations, then for the value 220 of the abscissa we will have $P(220) = 100/10\,000$). Fig. 7 shows that cooperation greatly improves the probability of finding quickly an optimal solution.

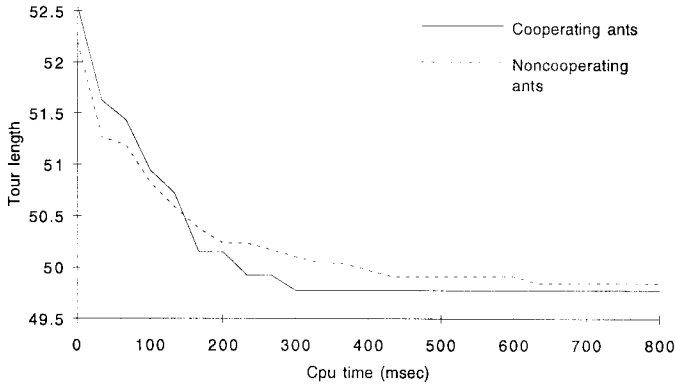


Fig. 8. Cooperating ants find better solutions in a shorter time. Test problem: CCAO [21]. Average on 25 runs. The number of ants was set to $m = 4$.

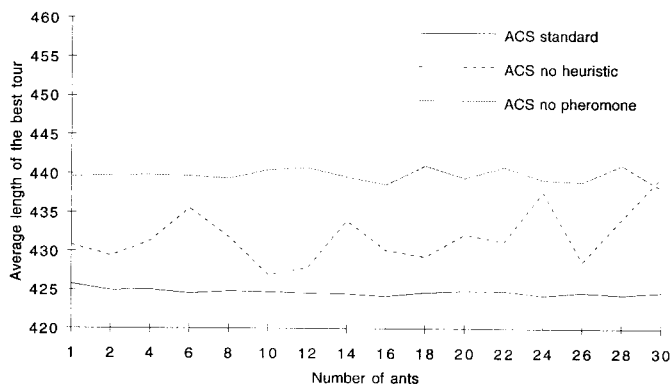


Fig. 9. Comparison between ACS standard, ACS with no heuristic (i.e., we set $\beta = 0$), and ACS in which ants neither sense nor deposit pheromone. Problem: Oliver30. Averaged over 30 trials, $10\,000/m$ iterations per trial.

In the second experiment (Fig. 8) the best solution found is plotted as a function of time (ms) for cooperating and noncooperating ants. The number of ants is fixed for both cases: $m = 4$. It is interesting to note that in the cooperative case, after 300 ms, the ACS always found the optimal solution, while noncooperating ants were not able to find it after 800 ms. During the first 150 ms (i.e., before the two lines in Fig. 8 cross) noncooperating ants outperform cooperating ants: good values of pheromone level are still being learned, and therefore the overhead due to pheromone updating is not yet compensated by the advantages which pheromone can provide in terms of directing the search toward good solutions.

D. The Importance of the Pheromone and the Heuristic Function

Experimental results have shown that the heuristic function η is fundamental in making the algorithm find good solutions in a reasonable time. In fact, when $\beta = 0$ ACS performance worsens significantly (see the ACS no heuristic graph in Fig. 9).

Fig. 9 also shows the behavior of ACS in an experiment in which ants neither sense nor deposit pheromone (ACS no pheromone graph). The result is that not using pheromone also deteriorates performance. This is a further confirmation of the results on the role of cooperation presented in Section IV-C.

TABLE II
COMPARISON OF ACS WITH OTHER HEURISTICS ON RANDOM INSTANCES OF THE SYMMETRIC TSP. COMPARISONS ON AVERAGE TOUR LENGTH OBTAINED ON FIVE 50-CITY PROBLEMS

Problem name	ACS (average)	SA (average)	EN (average)	SOM (average)
City set 1	5.88	5.88	5.98	6.06
City set 2	6.05	6.01	6.03	6.25
City set 3	5.58	5.65	5.70	5.83
City set 4	5.74	5.81	5.86	5.87
City set 5	6.18	6.33	6.49	6.70

The reason that ACS without the heuristic function performs better than ACS without pheromone is that in the first case, although not helped by heuristic information, the ACS is still guided by reinforcement provided by the global updating rule in the form of pheromone, while in the second case the ACS reduces to a stochastic multigreedy algorithm.

V. ACS: SOME COMPUTATIONAL RESULTS

We report on two sets of experiments. The first set compares the ACS with other heuristics. The choice of the test problems was dictated by published results found in the literature. The second set tests the ACS on some larger problems. Here the comparison is performed only with respect to the optimal or the best known result. The behavior of the ACS is excellent in both cases.

Most of the test problems can be found in TSPLIB: <http://www.iwr.uni-heidelberg.de/iwr/comopt/soft/TSPLIB95/TSPLIB.html>. When they are not available in this library we explicitly cite the reference where they can be found.

Given that during an iteration of the algorithm, each ant produces a tour, the total number of tours generated in the reported results is given by the number of iterations multiplied by the number of ants. The result of each trial is given by the best tour generated by the ants. Each experiment consists of at least 15 trials.

A. Comparison with Other Heuristics

To compare the ACS with other heuristics we consider two sets of TSP problems. The first set comprises five randomly generated 50-city problems, while the second set is composed of three geometric problems⁴ of between 50 and 100 cities. It is important to test the ACS on both random and geometric instances of the TSP because these two classes of problems have structural differences that can make them difficult for a particular algorithm and at the same time easy for another one.

Table II reports the results on the random instances. The heuristics with which we compare the ACS are simulated annealing (SA), elastic net (EN), and self-organizing map (SOM). Results on SA, EN, and SOM are from [13] and [34]. The ACS was run for 2500 iterations using ten ants (this amounts to approximately the same number of tours searched

⁴Geometric problems are problems taken from the real world (for example, they are generated choosing real cities and real distances).

TABLE III

COMPARISON OF ACS WITH OTHER HEURISTICS ON GEOMETRIC INSTANCES OF THE SYMMETRIC TSP. WE REPORT THE BEST INTEGER TOUR LENGTH, THE BEST REAL TOUR LENGTH (IN PARENTHESES), AND THE NUMBER OF TOURS REQUIRED TO FIND THE BEST INTEGER TOUR LENGTH (IN SQUARE BRACKETS). N/A MEANS "NOT AVAILABLE." IN THE LAST COLUMN THE OPTIMAL LENGTH IS AVAILABLE ONLY FOR INTEGER TOUR LENGTHS

Problem name	ACS	GA	EP	SA	Optimum
Eil50 (50-city problem)	425 (427.96) [1,830]	428 (N/A) [25,000]	426 (427.86) [100,000]	443 (N/A) [68,512]	425 (N/A)
Eil75 (75-city problem)	535 (542.37) [3,480]	545 (N/A) [80,000]	542 (549.18) [325,000]	580 (N/A) [173,250]	535 (N/A)
KroA100 (100-city problem)	21,282 (21,285.44) [4,820]	21,761 (N/A) [103,000]	N/A (N/A) [N/A]	N/A (N/A) [N/A]	21,282 (N/A)

by the heuristics with which we compare our results). The ACS results are averaged over 25 trials. The best average tour length for each problem is in boldface: the ACS almost always offers the best performance.

Table III reports the results on the geometric instances. The heuristics with which we compare the ACS in this case are a genetic algorithm (GA), evolutionary programming (EP), and simulated annealing (SA). The ACS is run for 1250 iterations using 20 ants (this amounts to approximately the same number of tours searched by the heuristics with which we compare our results). ACS results are averaged over 15 trials. In this case comparison is performed on the best results, as opposed to average results as in previous Table II (this choice was dictated by the availability of published results). The difference between integer and real tour length is that in the first case distances between cities are measured by integer numbers, while in the second case by floating point approximations of real numbers.

Results using EP are from [15], and those using GA are from [41] for Eil50 and Eil75, and from [6] for KroA100. Results using SA are from [29]. Eil50, Eil75 are from [14] and are included in TSPLIB with an additional city as Eil51.tsp and Eil76.tsp. KroA100 is also in TSPLIB. The best result for each problem is in boldface. Again, the ACS offers the best performance in nearly every case. Only for the Eil50 problem does it find a slightly worse solution using real-valued distance as compared with EP, but the ACS only visits 1830 tours, while EP used 100 000 such evaluations (although it is possible that EP found its best solution earlier in the run, this is not specified in the paper [15]).

B. ACS on Some Bigger Problems

When trying to solve big TSP problems it is common practice [28], [35] to use a data structure known as a *candidate list*. A candidate list is a list of preferred cities to be visited; it is a static data structure which contains, for a given city i , the cl closest cities, ordered by increasing distances; cl is a parameter that we set to $cl = 15$ in our experiments. We implemented therefore a version of the ACS [20] which incorporates a candidate list: an ant in this extended version of the ACS first chooses the city to move to among those

belonging to the candidate list. Only if none of the cities in the candidate list can be visited then it considers the rest of the cities. The ACS with the candidate list (see Table IV) was able to find good results for problems up to more than 1500 cities. The time to generate a tour grows only slightly more than linearly with the number of cities (this is much better than the quadratic growth obtained without the candidate list); on a Sun Sparc-server (50 MHz) it took approximately 0.02 s of CPU time to generate a tour for the d198 problem, 0.05 s for the pcb442, 0.07 s for the att532, 0.13 s for the rat783, and 0.48 s for the fl1577 (the reason for the more than linear increase in time is that the number of failures, that is, the number of times an ant has to choose the next city outside of the candidate list increases with the problem dimension).

VI. ACS PLUS LOCAL SEARCH

In Section V we have shown that the ACS is competitive with other nature-inspired algorithms on some relatively simple problems. On the other hand, in past years much work has been done to define ad-hoc heuristics (see [25] for an overview) to solve the TSP. In general, these ad-hoc heuristics greatly outperform, on the specific problem of the TSP, general purpose algorithms approaches like evolutionary computation and simulated annealing. Heuristic approaches to the TSP can be classified as tour constructive heuristics and tour improvement heuristics (these last also called local optimization heuristics). Tour constructive heuristics (see [4] for an overview) usually start selecting a random city from the set of cities and then incrementally build a feasible TSP solution by adding new cities chosen according to some heuristic rule. For example, the nearest neighbor heuristic builds a tour by adding the closest node in terms of distance from the last node inserted in the path. On the other hand, tour improvement heuristics start from a given tour and attempt to reduce its length by exchanging edges chosen according to some heuristic rule until a local optimum is found (i.e., until no further improvement is possible using the heuristic rule). The most used and well-known tour improvement heuristics are 2-opt and 3-opt [30], and Lin-Kernighan [31] in which, respectively, two, three, and a variable number of edges are exchanged. It has been experimentally shown [35] that, in general, tour improvement

TABLE IV

ACS PERFORMANCE FOR SOME BIGGER GEOMETRIC PROBLEMS (OVER 15 TRIALS). WE REPORT THE INTEGER LENGTH OF THE SHORTEST TOUR FOUND, THE NUMBER OF TOURS REQUIRED TO FIND IT, THE AVERAGE INTEGER LENGTH, THE STANDARD DEVIATION, THE OPTIMAL SOLUTION (FOR fl1577 WE GIVE, IN SQUARE BRACKETS, THE KNOWN LOWER AND UPPER BOUNDS, GIVEN THAT THE OPTIMAL SOLUTION IS NOT KNOWN), AND THE RELATIVE ERROR OF ACS

Problem name	ACS best integer length (1)	ACS number of tours generated to best	ACS average integer length	Standard deviation	Optimum (2)	Relative error $\frac{(1)-(2)}{(2)} \cdot 100$
d198 (198-city problem)	15,888	585,000	16,054	71	15,780	0.68 %
pcb442 (442-city problem)	51,268	595,000	51,690	188	50,779	0.96 %
att532 (532-city problem)	28,147	830,658	28,523	275	27,686	1.67 %
rat783 (783-city problem)	9,015	991,276	9,066	28	8,806	2.37 %
fl1577 (1577-city problem)	22,977	942,000	23,163	116	[22,204 – 22,249]	3.27÷3.48 %

heuristics produce better quality results than tour constructive heuristics. A general approach is to use tour constructive heuristics to generate a solution and then to apply a tour improvement heuristic to locally optimize it.

It has been shown recently [25] that it is more effective to alternate an improvement heuristic with mutations of the last (or of the best) solution produced, rather than iteratively executing a tour improvement heuristic starting from solutions generated randomly or by a constructive heuristic. An example of successful application of the above alternate strategy is the work by Freisleben and Merz [17], [18], in which a genetic algorithm is used to generate new solutions to be locally optimized by a tour improvement heuristic.

The ACS is a tour construction heuristic which, like Freisleben and Merz's genetic algorithm, produces a set of feasible solutions after each iteration which are in some sense a mutation of the previous best solution. It is, therefore, a reasonable guess that adding a tour improvement heuristic to the ACS could make it competitive with the best algorithms.

We therefore have added a tour improvement heuristic to the ACS. To maintain the ACS's ability to solve both TSP and ATSP problems we have decided to base the local optimization heuristic on a *restricted 3-opt* procedure [25], [27] that, while inserting/removing three edges on the path, considers only 3-opt moves that do not revert the order in which the cities are visited. In this case it is possible to change three edges on the tour (k, l) , (p, q) , and (r, s) with three other edges (k, q) , (p, s) , and (r, l) maintaining the previous orientations of all the other subtours. In case of ATSP problems, where in general $\delta(k, l) \neq \delta(l, k)$, this 3-opt procedure avoids unpredictable tour length changes due to the inversion of a subtour. In addition, when a candidate edge (k, l) to be removed is selected, the restricted 3-opt procedure restricts the search for the second edge (p, q) to be removed only to those edges such that $\delta(k, q) < \delta(k, l)$.

The implementation of the restricted 3-opt procedure includes some typical tricks which accelerate its use for TSP/ATSP problems. First, the search for the candidate nodes during the restricted 3-opt procedure is only made inside the candidate list [25]. Second, the procedure uses a data structure called *don't look bit* [4] in which each bit is associated to a node of the tour. At the beginning of the local optimization procedure all the bits are turned off and the bit associated to node r is turned on when a search for an improving move starting from r fails. The bit associated to node r is turned off again when a move involving r is performed. Third, only in the case of symmetric TSP's, while searching for 3-opt moves starting from a node r , the procedure also considers possible 2-opt moves with r as first node; the move executed is the best one among those proposed by 3-opt and by 2-opt. Last, a traditional array data structure to represent candidate lists and tours is used (see [16] for more sophisticated data structures).

ACS-3-opt also uses candidate lists in its constructive part; if there is no feasible node in the candidate list it chooses the closest node out of the candidate list (this is different from what happens in the ACS where, in case the candidate list contains no feasible nodes, then any of the remaining feasible cities can be chosen with a probability which is given by the normalized product of pheromone and closeness). This is a reasonable choice since most of the search performed by both the ACS and the local optimization procedure is made using edges belonging to the candidate lists. It is therefore pointless to direct search by using pheromone levels which are updated only very rarely.

A. Experimental Results

The experiments on ATSP problems presented in this section have been executed on a SUN Ultra1 SPARC Station (167 Mhz), while experiments on TSP problems on a SGI Challenge L server with eight 200 MHz CPU's, using only a single

```

Initialize
Loop /* at this level each loop is called an iteration */
  Each ant is positioned on a starting node
  Loop /* at this level each loop is called a step */
    Each ant applies a state transition rule to incrementally build a solution
    and a local pheromone updating rule
  Until all ants have built a complete solution
  Each ant is brought to a local minimum using a tour improvement
  heuristic based on 3-opt
  A global pheromone updating rule is applied
Until End_condition

```

Fig. 10. The ACS-3-opt algorithm.

TABLE V

RESULTS OBTAINED BY ACS-3-OPT ON ATSP PROBLEMS TAKEN FROM THE FIRST INTERNATIONAL CONTEST ON EVOLUTIONARY OPTIMIZATION [5]. WE REPORT THE LENGTH OF THE BEST TOUR FOUND BY ACS-3-OPT, THE CPU TIME USED TO FIND IT, THE AVERAGE LENGTH OF THE BEST TOUR FOUND AND THE AVERAGE CPU TIME USED TO FIND IT, THE OPTIMAL LENGTH, AND THE RELATIVE ERROR OF THE AVERAGE RESULT WITH RESPECT TO THE OPTIMAL SOLUTION

Problem name	ACS-3-opt best result (length)	ACS-3-opt best result (sec)	ACS-3-opt average (length)	ACS-3-opt average (sec)	Optimum	% Error
p43 (43-city problem)	2,810	1	2,810	2	2,810	0.00 %
ry48p (48-city problem)	14,422	2	14,422	19	14,422	0.00 %
ft70 (70-city problem)	38,673	3	38,679.8	6	38,673	0.02 %
kro124p (100-city problem)	36,230	3	36,230	25	36,230	0.00 %
ftv170* (170-city problem)	2,755	17	2,755	68	2,755	0.00 %

* ftv170 trials were run setting $cl=30$.

processor due to the sequential implementation of ACS-3-opt. For each test problem, ten trials have been executed. ACS-3-opt parameters were set to the following values (except if differently indicated): $m = 10$, $\beta = 2$, $q_0 = 0.98$, $\alpha = \rho = 0.1$, $\tau_0 = (n \cdot L_{nn})^{-1}$, and $cl = 20$.

Asymmetric TSP Problems: The results obtained with ACS-3-opt on ATSP problems are quite impressive. Experiments were run on the set of ATSP problems proposed in the First International Contest on Evolutionary Optimization [5] (see also <http://iridia.ulb.ac.be/langerman/ICEO.html>). For all the problems ACS-3-opt reached the optimal solution in a few seconds (see Table V) in all the ten trials, except in the case of ft70, a problem considered relatively hard, where the optimum was reached eight out of ten times.

In Table VI results obtained by ACS-3-opt are compared with those obtained by ATSP-GA [17], the winner of the ATSP competition. ATSP-GA is based on a genetic algorithm that starts its search from a population of individuals generated using a nearest neighbor heuristic. Individuals are strings of cities which represent feasible solutions. At each step two parents x and y are selected, and their edges are recombined using a procedure called DPX-ATSP. DPX-ATSP first deletes all edges in x that are not contained in y and then reconnects

the segments using a greedy heuristic based on a nearest neighbor choice. The new individuals are brought to the local optimum using a 3-opt procedure, and a new population is generated after the application of a mutation operation that randomly removes and reconnects some edges in the tour.

The 3-opt procedure used by ATSP-GA is very similar to our restricted 3-opt, which makes the comparison between the two approaches straightforward. ACS-3-opt outperforms ATSP-GA in terms of both closeness to the optimal solution and of CPU time used. Moreover, ATSP-GA experiments have been performed using a DEC Alpha Station (266 MHz), a machine faster than our SUN Ultra1 SPARC Station.

Symmetric TSP Problems: If we now turn to symmetric TSP problems, it turns out that STSP-GA (STSP-GA experiments have been performed using a 175 MHz DEC Alpha Station), the algorithm that won the First International Contest on Evolutionary Optimization in the symmetric TSP category, outperforms ACS-3-opt (see Tables VII and VIII). The results used for comparisons are those published in [18], which are slightly better than those published in [17].

Our results, on the other hand, are comparable to those obtained by other algorithms considered to be very good. For example, on the lin318 problem ACS-3-opt has approximately

TABLE VI

COMPARISON BETWEEN ACS-3-OPT AND ATSP-GA ON ATSP PROBLEMS TAKEN FROM THE FIRST INTERNATIONAL CONTEST ON EVOLUTIONARY OPTIMIZATION [5]. WE REPORT THE AVERAGE LENGTH OF THE BEST TOUR FOUND, THE AVERAGE CPU TIME USED TO FIND IT, AND THE RELATIVE ERROR WITH RESPECT TO THE OPTIMAL SOLUTION FOR BOTH APPROACHES

Problem name	ACS-3-opt average (length)	ACS-3-opt average (sec)	ACS-3-opt % error	ATSP-GA average (length)	ATSP-GA average (sec)	ATSP-GA % error
p43 (43-city problem)	2,810	2	0.00 %	2,810	10	0.00 %
ry48p (48-city problem)	14,422	19	0.00 %	14,440	30	0.12 %
ft70 (70-city problem)	38,679.8	6	0.02 %	38,683.8	639	0.03 %
kro124p (100-city problem)	36,230	25	0.00 %	36,235.3	115	0.01 %
ftv170 (170-city problem)	2,755	68	0.00 %	2,766.1	211	0.40 %

TABLE VII

RESULTS OBTAINED BY ACS-3-OPT ON TSP PROBLEMS TAKEN FROM THE FIRST INTERNATIONAL CONTEST ON EVOLUTIONARY OPTIMIZATION [5]. WE REPORT THE LENGTH OF THE BEST TOUR FOUND BY ACS-3-OPT, THE CPU TIME USED TO FIND IT, THE AVERAGE LENGTH OF THE BEST TOUR FOUND AND THE AVERAGE CPU TIME USED TO FIND IT, AND THE OPTIMAL LENGTH AND THE RELATIVE ERROR OF THE AVERAGE RESULT WITH RESPECT TO THE OPTIMAL SOLUTION

Problem name	ACS-3-opt best result (length)	ACS-3-opt best result (sec)	ACS-3-opt average (length)	ACS-3-opt average (sec)	Optimum	% Error
d198 (198-city problem)	15,780	16	15,781.7	238	15,780	0.01 %
lin318* (318-city problem)	42,029	101	42,029	537	42,029	0.00 %
att532 (532-city problem)	27,693	133	27,718.2	810	27,686	0.11 %
rat783 (783-city problem)	8,818	1,317	8,837.9	1,280	8,806	0.36 %

* lin318 trials were run setting $q_0=0.95$.

TABLE VIII

COMPARISON BETWEEN ACS-3-OPT AND STSP-GA ON TSP PROBLEMS TAKEN FROM THE FIRST INTERNATIONAL CONTEST ON EVOLUTIONARY OPTIMIZATION [5]. WE REPORT THE AVERAGE LENGTH OF THE BEST TOUR FOUND, THE AVERAGE CPU TIME USED TO FIND IT, AND THE RELATIVE ERROR WITH RESPECT TO THE OPTIMAL SOLUTION FOR BOTH APPROACHES

Problem name	ACS-3-opt average (length)	ACS-3-opt average (sec)	ACS-3-opt % error	STSP-GA average (length)	STSP-GA average (sec)	STSP-GA % error
d198 (198-city problem)	15,781.7	238	0.01 %	15,780	253	0.00 %
lin318 (318-city problem)	42,029	537	0.00 %	42,029	2,054	0.00 %
att532 (532-city problem)	27,718.2	810	0.11 %	27,693.7	11,780	0.03 %
rat783 (783-city problem)	8,837.9	1,280	0.36 %	8,807.3	21,210	0.01 %

the same performance as the “large step Markov chain algorithm” [33]. This algorithm is based on a simulated annealing mechanism that uses as improvement heuristic a restricted 3-opt heuristic very similar to ours (the only difference is that they do not consider 2-opt moves) and a mutation procedure called *double-bridge*. (The *double-bridge* mutation has the

property that it is the smallest change (four edges) that can not be reverted in one step by 3-opt, LK, and 2-opt.)

A fair comparison of our results with the results obtained with the currently best performing algorithms for symmetric TSP’s [25] is difficult since they use as local search a Lin–Kernighan heuristic based on a segment-tree data structure

[16] that is faster and gives better results than our restricted-3-opt procedure. It will be the subject of future work to add such a procedure to the ACS.

VII. DISCUSSION AND CONCLUSIONS

An intuitive explanation of how the ACS works, which emerges from the experimental results presented in the preceding sections, is as follows. Once all the ants have generated a tour, the best ant deposits (at the end of iteration t) its pheromone, defining in this way a “preferred tour” for search in the following algorithm iteration $t + 1$. In fact, during iteration $t + 1$ ants will see edges belonging to the best tour as highly desirable and will choose them with high probability. Still, guided exploration [see (3) and (1)] together with the fact that local updating “eats” pheromone away (i.e., it diminishes the amount of pheromone on visited edges, making them less desirable for future ants) allows for the search of new, possibly better tours in the neighborhood⁵ of the previous best tour. So the ACS can be seen as a sort of guided parallel stochastic search in the neighborhood of the best tour.

Recently there has been growing interest in the application of ant colony algorithms to difficult combinatorial problems. A first example is the work of Schoonderwoerd *et al.* [37], who apply an ant colony algorithm to the load balancing problem in telecommunications networks. Their algorithm takes inspiration from the same biological metaphor as ant system, although their implementation differs in many details due to the different characteristics of the problem. Another interesting ongoing research is that of Stützle and Hoos who are studying various extensions of ant system to improve its performance: in [38] they impose an upper and lower bound on the value of pheromone on edges, and in [39] they add local search, much in the same spirit as we did in the previous Section VI.

Besides the two works above, among the “nature-inspired” heuristics, the closest to the ACS seems to be Baluja and Caruana’s population based incremental learning (PBIL) [1]. PBIL, which takes inspiration from genetic algorithms, maintains a vector of real numbers, the generating vector, which plays a role similar to that of the population in GA’s. Starting from this vector, a population of binary strings is randomly generated; each string in the population will have the i th bit set to one with a probability which is a function of the i th value in the generating vector (in practice, values in the generating vector are normalized to the interval $[0, 1]$ so that they can directly represent the probabilities). Once a population of solutions is created, the generated solutions are evaluated, and this evaluation is used to increase (or decrease) the probabilities in the generating vector so that good (bad) solutions in the future generations will be produced with higher (lower) probability. When applied to TSP, PBIL uses the following encoding: a solution is a string of size $n \log_2 n$ bits, where n is the number of cities; each city is assigned a string of length $\log_2 n$ which is interpreted as an integer. Cities are then ordered by increasing integer values; in case of ties the left-most city in the string

comes first in the tour. In the ACS, the pheromone matrix plays a role similar to Baluja’s generating vector, and pheromone updating has the same goal as updating the probabilities in the generating vector. Still, the two approaches are very different since in the ACS the pheromone matrix changes while ants build their solutions, while in PBIL the probability vector is modified only after a population of solutions has been generated. Moreover, the ACS uses heuristic to direct search, while PBIL does not.

There are a number of ways in which the ant colony approach can be improved and/or changed. A first possibility regards the number of ants which should contribute to the global updating rule. In ant system all the ants deposited their pheromone, while in the ACS only the best one does: obviously there are intermediate possibilities. Baluja and Caruana [1] have shown that the use of the two best individuals can help PBIL to obtain better results, since the probability of being trapped in a local minimum becomes smaller. Another change to the ACS could be, again taking inspiration from [1], allowing ants which produce very bad tours to subtract pheromone.

A second possibility is to move from the current parallel local updating of pheromone to a sequential one. In the ACS all ants apply the local updating rule in parallel, while they are building their tours. We could imagine a modified ACS in which ants build tours sequentially: the first ant starts, builds its tour, and as a side effect, changes the pheromone on visited edges. Then the second ant starts and so on until the last of the m ants has built its tour. At this point the global updating rule is applied. This scheme will determine a different search regime, in which the preferred tour will tend to remain the same for all the ants (as opposed to the situation in the ACS, in which local updating shuffles the tours). Nevertheless, the search will be diversified since the first ants in the sequence will search in a neighborhood of the preferred tour that is more narrow than later ones (in fact, pheromone on the preferred tour decreases as more ants eat it away, making the relative desirability of edges of the preferred tour decrease). The role of local updating in this case would be similar to that of the temperature in simulated annealing, with the main difference that here the temperature changes during each algorithm iteration.

Last, it would be interesting to add to the ACS a more effective local optimizer than that used in Section VI. A possibility we will investigate in the near future is the use of the Lin–Kernighan heuristic.

Another interesting subject of ongoing research is to establish the class of problems that can be attacked by the ACS. Our intuition is that ant colony algorithms can be applied to combinatorial optimization problems by defining an appropriate graph representation of the problem considered and a heuristic that guides the construction of feasible solutions. Then, artificial ants much like those used in the TSP application presented in this paper can be used to search for good solutions of the problem under consideration. The above intuition is supported by encouraging, although preliminary, results obtained with ant system on the quadratic assignment problem [12], [32], [42].

⁵The form of the neighborhood is given by the previous history of the system, that is, by pheromone accumulated on edges.

In conclusion, in this paper we have shown that the ACS is an interesting novel approach to parallel stochastic optimization of the TSP. The ACS has been shown to compare favorably with previous attempts to apply other heuristic algorithms like genetic algorithms, evolutionary programming, and simulated annealing. Nevertheless, competition on the TSP is very tough, and a combination of a constructive method which generates good starting solution with local search which takes these solutions to a local optimum seems to be the best strategy [25]. We have shown that the ACS is also a very good constructive heuristic to provide such starting solutions for local optimizers.

APPENDIX A THE ACS ALGORITHM

```

1)/* Initialization phase */
For each pair  $(r, s) \tau(r, s) := \tau_0$  End-for
For  $k := 1$  to  $m$  do
  Let  $r_{k1}$  be the starting city for ant  $k$ 
   $J_k(r_{k1}) := \{1, \dots, n\} - r_{k1}$ 
  /*  $J_k(r_{k1})$  is the set of yet to be visited cities for
  ant  $k$  in city  $r_{k1}$  */
   $r_k := r_{k1}$  /*  $r_k$  is the city where ant
   $k$  is located */
End-for
2)/* This is the phase in which ants build their tours.
The tour of ant  $k$  is stored in  $Tour_k$ . */
For  $i := 1$  to  $n$  do
  If  $i < n$ 
  Then
    For  $k := 1$  to  $m$  do
      Choose the next city  $s_k$  according to (3) and (1)
       $J_k(s_k) := J_k(r_k) - s_k$ 
       $Tour_k(i) := (r_k, s_k)$ 
    End-for
  Else
    For  $k := 1$  to  $m$  do
      /* In this cycle all the ants go back to the initial
      city  $r_{k1}$  */
       $s_k := r_{k1}$ 
       $Tour_k(i) := (r_k, s_k)$ 
    End-for
  End-if
/* In this phase local updating occurs and pheromone is
updated using (5)*/
For  $k := 1$  to  $m$  do
   $\tau(r_k, s_k) := (1 - \rho)\tau(r_k, s_k) + \rho\tau_0$ 
   $r_k := s_k$  /* New city for ant  $k$  */
End-for
End-for
3)/* In this phase global updating occurs and pheromone is
updated */
For  $k := 1$  to  $m$  do
  Compute  $L_k$  /*  $L_k$  is the length of the tour done by
  ant  $k$  */
End-for

```

```

Compute  $L_{best}$ 
/*Update edges belonging to  $L_{best}$  using (4) */
For each edge  $(r, s)$ 
   $\tau(r_k, s_k) := (1 - \alpha)\tau(r_k, s_k) + \alpha (L_{best})^{-1}$ 
End-for
4) If (End.condition = True)
  then Print shortest of  $L_k$ 
  else goto Phase 2

```

ACKNOWLEDGMENT

The authors wish to thank N. Bradshaw, G. Di Caro, D. Fogel, and five anonymous referees for their precious comments on a previous version of this article.

REFERENCES

- [1] S. Baluja and R. Caruana, "Removing the genetics from the standard genetic algorithm," in *Proc. ML-95, 12th Int. Conf. Machine Learning*. Palo Alto, CA: Morgan Kaufmann, 1995, pp. 38–46.
- [2] A. G. Barto, R. S. Sutton, and P. S. Brower, "Associative search network: A reinforcement learning associative memory," *Biological Cybern.*, vol. 40, pp. 201–211, 1981.
- [3] R. Beckers, J. L. Deneubourg, and S. Goss, "Trails and U-turns in the selection of the shortest path by the ant *Lasius Niger*," *J. Theoretical Biology*, vol. 159, pp. 397–415, 1992.
- [4] J. L. Bentley, "Fast algorithms for geometric traveling salesman problems," *ORSA J. Comput.*, vol. 4, pp. 387–411, 1992.
- [5] H. Bersini, M. Dorigo, S. Langerman, G. Seront, and L. M. Gambardella, "Results of the first international contest on evolutionary optimization (1st ICEO)," in *Proc. IEEE Int. Conf. Evolutionary Computation, IEEE-EC 96*, pp. 611–615.
- [6] H. Bersini, C. Oury, and M. Dorigo, "Hybridization of genetic algorithms," Université Libre de Bruxelles, Belgium, Tech. Rep. IRIDIA/95-22, 1995.
- [7] A. Colomi, M. Dorigo, and V. Maniezzo, "Distributed optimization by ant colonies," in *Proc. ECAL91—Eur. Conf. Artificial Life*. New York: Elsevier, 1991, pp. 134–142.
- [8] ———, "An investigation of some properties of an ant algorithm," in *Proc. Parallel Problem Solving from Nature Conference (PPSN 92)*. New York: Elsevier, 1992, pp. 509–520.
- [9] J. L. Deneubourg, "Application de l'ordre par fluctuations à la description de certaines étapes de la construction du nid chez les termites," *Insect Sociaux*, vol. 24, pp. 117–130, 1977.
- [10] M. Dorigo, "Optimization, learning and natural algorithms," Ph.D. dissertation, DEI, Politecnico di Milano, Italy, 1992 (in Italian).
- [11] M. Dorigo and L. M. Gambardella, "A study of some properties of Ant-Q," in *Proc. PPSN IV—4th Int. Conf. Parallel Problem Solving from Nature*. Berlin, Germany: Springer-Verlag, 1996, pp. 656–665.
- [12] M. Dorigo, V. Maniezzo, and A. Colomi, "The ant system: Optimization by a colony of cooperating agents," *IEEE Trans. Syst. Man, Cybern. B*, vol. 26, no. 2, pp. 29–41, 1996.
- [13] R. Durbin and D. Willshaw, "An analogue approach to the travelling salesman problem using an elastic net method," *Nature*, vol. 326, pp. 689–691, 1987.
- [14] S. Eilon, C. D. T. Watson-Gandy, and N. Christofides, "Distribution management: Mathematical modeling and practical analysis," *Oper. Res. Quart.*, vol. 20, pp. 37–53, 1969.
- [15] D. B. Fogel, "Applying evolutionary programming to selected traveling salesman problems," *Cybern. Syst. Int. J.*, vol. 24, pp. 27–36, 1993.
- [16] M. L. Fredman, D. S. Johnson, L. A. McGeoch, and G. Ostheimer, "Data structures for traveling salesmen," *J. Algorithms*, vol. 18, pp. 432–479, 1995.
- [17] B. Freisleben and P. Merz, "Genetic local search algorithm for solving symmetric and asymmetric traveling salesman problems," in *Proc. IEEE Int. Conf. Evolutionary Computation, IEEE-EC 96*, 1996, pp. 616–621.
- [18] ———, "New genetic local search operators for the traveling salesman problem," in *Proc. PPSN IV—4th Int. Conf. Parallel Problem Solving from Nature*. Berlin, Germany: Springer-Verlag, 1996, pp. 890–899.
- [19] L. M. Gambardella and M. Dorigo, "Ant-Q: A reinforcement learning approach to the traveling salesman problem," in *Proc. ML-95, 12th Int. Conf. Machine Learning*. Palo Alto, CA: Morgan Kaufmann, 1995, pp. 252–260.

- [20] ———, "Solving symmetric and asymmetric TSP's by ant colonies," *Proc. IEEE Int. Conf. Evolutionary Computation, IEEE-EC 96*, 1996, pp. 622–627.
- [21] B. Golden and W. Stewart, "Empiric analysis of heuristics," in *The Traveling Salesman Problem*, E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy-Kan, and D. B. Shmoys, Eds. New York: Wiley, 1985.
- [22] S. Goss, S. Aron, J. L. Deneubourg, and J. M. Pasteels, "Self-organized shortcuts in the argentine ant," *Naturwissenschaften*, vol. 76, pp. 579–581, 1989.
- [23] P. P. Grassé, "La reconstruction du nid et les coordinations inter-individuelles chez *Bellicositermes natalensis* et *Cubitermes* sp. La théorie de la stigmergie: Essai d'interprétation des termites constructeurs," *Insect Sociaux*, vol. 6, pp. 41–83, 1959.
- [24] B. Hölldobler and E. O. Wilson, *The Ants*. Berlin: Springer-Verlag, 1990.
- [25] D. S. Johnson and L. A. McGeoch, "The travelling salesman problem: a case study in local optimization," in *Local Search in Combinatorial Optimization*, E. H. L. Aarts and J. K. Lenstra, Eds. New York: Wiley: New York, 1997.
- [26] L. P. Kaelbling, L. M. Littman, and A. W. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, pp. 237–285, 1996.
- [27] P.-C. Kanellakis and C. H. Papadimitriou, "Local search for the asymmetric traveling salesman problem," *Oper. Res.*, vol. 28, no. 5, pp. 1087–1099, 1980.
- [28] E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy-Kan, and D. B. Shmoys, Eds., *The Traveling Salesman Problem*. New York: Wiley, 1985.
- [29] F.-T. Lin, C.-Y. Kao, and C.-C. Hsu, "Applying the genetic approach to simulated annealing in solving some NP-Hard problems," *IEEE Trans. Syst., Man, Cybern.*, vol. 23, pp. 1752–1767, 1993.
- [30] S. Lin, "Computer solutions of the traveling salesman problem," *Bell Syst. J.*, vol. 44, pp. 2245–2269, 1965.
- [31] S. Lin and B. W. Kernighan, "An effective heuristic algorithm for the traveling salesman problem," *Oper. Res.*, vol. 21, pp. 498–516, 1973.
- [32] V. Maniezzo, A. Colorni, and M. Dorigo, "The ant system applied to the quadratic assignment problem," Université Libre de Bruxelles, Belgium, Tech. Rep. IRIDIA/94-28, 1994.
- [33] O. Martin, S. W. Otto, and E. W. Felten, "Large-step Markov chains for the TSP incorporating local search heuristics," *Oper. Res. Lett.*, vol. 11, pp. 219–224, 1992.
- [34] J.-Y. Potvin, "The traveling salesman problem: A neural network perspective," *ORSA J. Comput.*, vol. 5, no. 4, pp. 328–347, 1993.
- [35] G. Reinelt, *The Traveling Salesman: Computational Solutions for TSP Applications*. New York: Springer-Verlag, 1994.
- [36] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, "An analysis of several heuristics for the traveling salesman problem," *SIAM J. Comput.*, vol. 6, pp. 563–581, 1977.
- [37] R. Schoonderwoerd, O. Holland, J. Bruten, and L. Rothkrantz, "Ant-based load balancing in telecommunications networks," *Adaptive Behavior*, vol. 5, no. 2, 1997.
- [38] T. Stützle and H. Hoos, "Improvements on the ant system, introducing the MAX-MIN ant system," in *Proc. ICANNGA97—Third Int. Conf. Artificial Neural Networks and Genetic Algorithms*. Wien, Germany: Springer-Verlag, 1997.
- [39] ———, "The MAX-MIN ant system and local search for the traveling salesman problem," in *Proc. ICEC'97—1997 IEEE 4th Int. Conf. Evolutionary Computation*, 1997, pp. 309–314.
- [40] C. J. C. H. Watkins, "Learning with delayed rewards," Ph.D. dissertation, Psychology Dept., Univ. of Cambridge, UK, 1989.
- [41] D. Whitley, T. Starkweather, and D. Fuquay, "Scheduling problems and travelling salesman: the genetic edge recombination operator," in *Proc. the 3rd Int. Conf. Genetic Algorithms*. Palo Alto, CA: Morgan Kaufmann, 1989, pp. 133–140.
- [42] L. M. Gambardella, E. Taillard, and M. Dorigo, "Ant colonies for QAP," IDSIA, Lugano, Switzerland, Tech. Rep. IDSIA 97-4, 1997.



Marco Dorigo (S'92–M'92–SM'96) was born in Milan, Italy, in 1961. He received the Laurea (Master of Technology) degree in industrial technologies engineering in 1986 and the Ph.D. degree in information and systems electronic engineering in 1992 from Politecnico di Milano, Milan, Italy, and the title of Agrégé de l'Enseignement Supérieur, from the Université Libre de Bruxelles, Belgium, in 1995.

From 1992 to 1993 he was a Research Fellow at the International Computer Science Institute of Berkeley, CA. In 1993 he was a NATO-CNR Fellow and from 1994 to 1996 a Marie Curie Fellow. Since 1996 he has been a Research Associate with the FNRS, the Belgian National Fund for Scientific Research. His research areas include evolutionary computation, distributed models of computation, and reinforcement learning. He is interested in applications to autonomous robotics, combinatorial optimization, and telecommunications networks.

Dr. Dorigo is an Associate Editor for the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS and for the IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION. He is a member of the Editorial Board of *Evolutionary Computation* and of *Adaptive Behavior*. He was awarded the 1996 Italian Prize for Artificial Intelligence. He is a member of the Italian Association for Artificial Intelligence (AI*IA).



Luca Maria Gambardella (M'91) was born in Saronno, Italy, in 1962. He received the Laurea degree in computer science in 1985 from the Università degli Studi di Pisa, Facoltà di Scienze Matematiche Fisiche e Naturali.

Since 1988 he has been Research Director at IDSIA, Istituto Dalle Molle di Studi sull'Intelligenza Artificiale, a private research institute located in Lugano, Switzerland, supported by Canton Ticino and Swiss Confederation. His major research interests are in the area of machine learning and adaptive systems applied to robotics and optimization problems. He is leading several research and industrial projects in the area of collective robotics, cooperation and learning for combinatorial optimization, scheduling, and simulation supported by the Swiss National Foundation and by the Swiss Technology and Innovation Commission.